

Programación con *sockets* - Sockets UDP

El protocolo UDP, tal y como hemos estudiado desde una perspectiva teórica, nos ofrece un servicio de transporte **no orientado a conexión**. Como programadores de aplicaciones que se comunican por redes, para nosotros esto implica que no existirá una llamada a `connect()` por parte del cliente, ni tampoco se usará la función `accept()` en el servidor. Estas, como aprenderemos en la práctica siguiente, son las funciones clave para solicitar y responder a las peticiones de conexión.

En realidad, UDP ofrece una funcionalidad muy ligera sobre el uso de los *sockets* IP. Cada paquete tendrá asociada una dirección IP de origen y una de destino, ya sean IPv4 o IPv6. Al operar con segmentos UDP, además, aparece un concepto que no se ha usado en las prácticas previas: el **número de puerto**.

Nota

Como ya sabemos, el número de puerto en la capa de transporte permite vincular **cada conexión de red con un proceso** en ejecución en el *host*. En general, usaremos números de puerto asociados al intercambio de datagramas con el protocolo UDP o a cualquier conexión TCP entre un cliente y un servidor (tema tratado en el cuarto cuaderno de prácticas sobre programación con *sockets*.)

Como es lógico, para enviar y recibir segmentos UDP precisaremos operar con direcciones. Por ello serán esenciales todas las nociones adquiridas en las anteriores prácticas sobre *sockets*, en especial las relacionadas con resolución y conversión de direcciones.

El primer paso será invocar a la función `socket()` para crear el *endpoint* o punto de conexión. Este será utilizado para enviar (escribir) o recibir (leer) información por ese *socket*. **Según el tipo de IP** asociada al *socket*, el segmento UDP se enviará a un solo destinatario (*unicast*), a todos los equipos de una red (*broadcast*) o a un subconjunto de ellos (*multicast*).

Esquema de pasos e interacción

Los pasos a seguir para crear un *socket* UDP y, a continuación, enviar o recibir información a través de él, son los representados esquemáticamente en el Diagrama 1. En la parte superior se representa el cliente y en la inferior el servidor, con la **interacción entre ambos**.

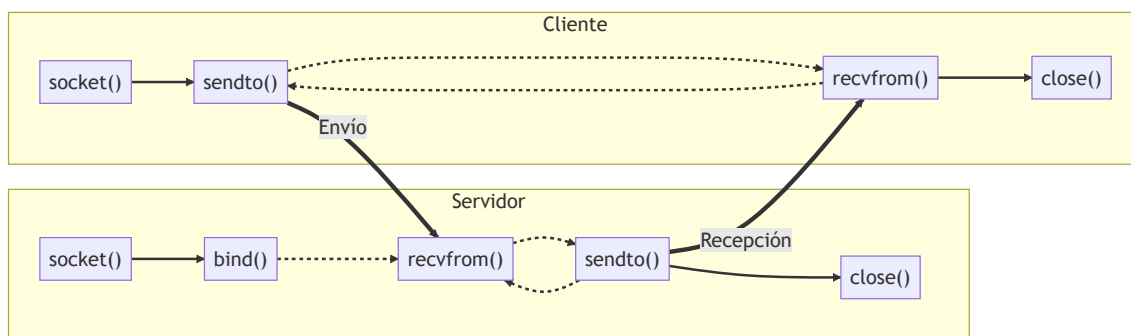


Diagrama 1. Secuencia de ejecución de un cliente y un servidor que usan el protocolo UDP.

Tenemos llamadas a las funciones encargadas de la creación del *socket*: función `socket()`; el envío de un paquete: función `sendto()`; la recepción de un paquete: función `recvfrom()`, y el cierre: función `close()`, a las que hay que añadir la función `bind()` en el servidor, antes de poder recibir datos desde el cliente. Dicha función vincula el proceso del servidor con un puerto UDP concreto.

Inicialización del servidor y el cliente

Un cliente puede enviar segmentos UDP sin necesidad de establecer una conexión previa, algo indispensable en el caso de TCP (lo estudiaremos en la siguiente práctica), por lo que no se precisa más inicialización que la apertura del *socket* previa al envío. El servidor, por el contrario, sí **ha de configurarse apropiadamente** antes de estar en disposición de recibir paquetes. Para ello seguirá los pasos indicados a continuación:

- `socket()`: la creación del *socket* determinará, con el primer parámetro, el tipo de dirección IP que se empleará. El tipo de *socket* (segundo argumento) será `SOCK_DGRAM` y el tercero `IPPROTO_UDP`.

Nota

Al invocar a `socket()` con `SOCK_DGRAM` como segundo argumento, el tercer parámetro puede ser `0` ya que se asume que, para ese tipo de *socket*, el protocolo asociado es UDP. En cualquier caso, lo habitual es que los argumentos para dicha función se obtengan de manera automática mediante la función `getaddrinfo()` (revisa el guion de la práctica dedicada a las direcciones para recordar el uso de esta).

- `bind()`: un programa que quiera recibir los datos entrantes a través de un puerto UDP, ha de *enlazarse* con dicho puerto y la dirección IP de la interfaz correspondiente, para lo cual facilitará los parámetros adecuados a esta función.

Dado que el cliente también usará la función `socket()` para crear el *socket*, con parámetros análogos a los del servidor, la diferencia fundamental entre ambos programas estriba en que el servidor usa la función `bind()` para vincularse a una determinada interfaz de red y puerto.

Intercambio de datos

Completada la inicialización, y tal y como denota el esquema del **Diagrama 1**, será el cliente el que inicie el proceso de intercambio de información, enviando un paquete de datos al servidor. Este, mientras tanto, habrá quedado a la espera al invocar a la función `recvfrom()`, al ser este un servicio *bloqueante* por defecto.

Se asume que el cliente **conoce la dirección IP y el puerto** al que se ha vinculado el servidor, datos que facilitará a la función `sendto()`. A esta se entregarán, entre otros parámetros, el contenido del bloque de datos a enviar (*payload* del segmento UDP) y su longitud.

Cuando el paquete llega al servidor, su contenido es devuelto por la función `recvfrom()`. Esta facilita, por una parte, los datos enviados por el cliente mediante `sendto()`. Por otra, también se entrega en el penúltimo parámetro la **dirección IP y el puerto del cliente** que ha enviado el paquete. Esta información es imprescindible en caso de que el servidor necesite responder al cliente.

La función `recvfrom()`

Con esta función se obtiene un paquete de datos entrante en alguna de las interfaces de red. Al invocarla deben facilitarse los siguientes argumentos:

- **socket:** el valor devuelto por la función `socket()` y que identifica al *socket* del que se leerá.
- **paquete:** un puntero `void*` al espacio de memoria que, reservado previamente, servirá para almacenar los datos obtenidos.
- **tamaño:** un entero que comunica a la función el espacio disponible para almacenar datos leídos.
- **modificadores:** la unión de un conjunto de modificadores para la operación de lectura. Puede tomar el valor `0`. Los detalles sobre estos modificadores se encuentran en la documentación electrónica.
- **origen:** un puntero `struct sockaddr*` que servirá para devolver información sobre el origen del paquete de datos obtenido.
- **long_direccion:** un puntero `int*` al que habrá que asignar el tamaño de la estructura `sockaddr` anterior y que, tras la llamada, contendrá el tamaño real de la dirección devuelta.

El valor devuelto por `recvfrom()` indica el tamaño en bytes del paquete de datos leído. De ser `-1` indicaría un error. Ya sabemos qué hacer en este caso: obtener el código de error de la variable `errno` y tratarlo como proceda según nuestro programa.

La función `sendto()`

Esta función envía un paquete de datos a un destino concreto, cuya dirección IP y puerto se entregan como parámetros. La lista de argumentos a facilitar a esta función es la siguiente:

- **socket:** el valor devuelto por la función `socket()` y que identifica el *socket* en el que se escribirá.
- **paquete:** un puntero de tipo `void*` al paquete de datos que se pretende enviar.
- **tamaño:** el tamaño en bytes del anterior paquete de datos.
- **modificadores:** al igual que con la función `recvfrom()`, los valores para este parámetro se detallan en la documentación. En principio lo dejaremos a `0`.
- **destino:** un puntero `struct sockaddr*` con la dirección a la que se enviará el paquete.
- **long_direccion:** un valor entero sin signo que indica el tamaño de la dirección facilitada como parámetro previo. Esto permite emplear direcciones IPv4 e IPv6, entre otras posibilidades.

Nota

El valor devuelto por esta función indicará el número de bytes efectivamente enviados. En caso de error se devolverá `-1`, como ocurría con `recvfrom()`.

Desarrollo de un servidor UDP

Una vez que conocemos los pasos y funciones precisas para comunicar dos procesos mediante UDP, veamos en la práctica cómo implementar un servidor y un cliente. Comenzamos con el servidor, cuya finalidad será la de **hacer eco de la información que reciba**, es decir, devolver el mismo paquete de datos al origen.

Función principal del servidor

Comencemos escribiendo la función `main()` de nuestro servidor. Desde el cuerpo de esta se llamará a otras que serán descritas en los siguientes apartados. El código de dicha función principal es el siguiente:

```
int main(int argc, char *argv[]) {
    const auto PUERTO_ECO = "32768";
    struct sockaddr_storage dircliente;
    socklen_t longdircliente;
    char *paquete = new char[IP_MAXPACKET];

    int misocket = configura_servidor(PUERTO_ECO), socketcliente;

    while(true) {
        longdircliente = sizeof dircliente;
        auto bytes = recvfrom(misocket, paquete, IP_MAXPACKET, 0,
                               (struct sockaddr *) &dircliente, &longdircliente);
        if(bytes == -1) ERROR("recvfrom()");

        paquete[bytes] = '\0';
        informa_paquete(paquete, dircliente);

        sendto(misocket, paquete, bytes, MSG_WAITALL,
               (struct sockaddr *) &dircliente, longdircliente);
    }

    close(misocket);
    return 0;
}
```

La variable `paquete`, un vector de caracteres con el tamaño máximo indicado por la constante `IP_MAXPACKET`, nos servirá para almacenar los segmentos UDP recibidos y también preparar las respuestas. El *socket* que usaremos para comunicarnos se almacenará en la variable `misocket`, vinculado al **puerto UDP 32768**. El valor de dicha variable es obtenido de la función `configura_servidor()` descrita después. De esta manera *aislamos* (desacoplamos) la complejidad de configuración del *socket* del cuerpo principal del programa.

El código útil de esta función se encuentra en un bucle sin fin, de forma que las acciones indicadas a continuación se repitan una y otra vez:

1. Recepción de un segmento UDP mediante la función `recvfrom()`. Esta facilita asimismo, en una estructura `sockaddr`, los datos del *host* que envía dicho segmento, de los cuales nos servirán su dirección IP (tipo y dirección propiamente dicha) y puerto a fin de poder enviar una respuesta.
2. Los datos devueltos por `recvfrom()` se envían, tras agregar un byte `'\0'` al final, a la función `informa_paquete()` para que esta notifique por la consola la actividad de peticiones de eco recibidas.

3. Por último, y usando exactamente los mismos parámetros que hemos recibido de la función `recvfrom()` (al fin y al cabo queremos generar un eco), usamos la función `sendto()` para devolver el segmento a su origen.

Nota

En esta versión inicial del programa no se sale nunca del bucle `while`, por lo que el *socket* del servidor no se cerraría hasta en tanto no interrumpamos la ejecución del programa.

La función de configuración

La configuración de un *socket* UDP para un programa que quiere quedar a la escucha, actuando como lo haría un servidor, se compone de un conjunto de pasos que es siempre el mismo. El código encargado de completarlos es el mostrado a continuación:

```
#define ERROR(funcion) throw runtime_error(funcion + string(strerror(errno)))

int configura_servidor(const char *puerto) {
    struct addrinfo conf {
        .ai_flags = AI_PASSIVE,
        .ai_family = AF_UNSPEC,
        .ai_socktype = SOCK_DGRAM
    }, *resultado;

    if(getaddrinfo(NULL, puerto, &conf, &resultado)) ERROR("getaddrinfo()");

    int unsocket = -1;
    while(unsocket = socket(resultado->ai_family,
        resultado->ai_socktype, resultado->ai_protocol)) {
        if(unsocket != -1) break;

        cout << "socket() - " + string(strerror(errno)) << endl;
        resultado = resultado->ai_next;
    }

    if(bind(unsocket, resultado->ai_addr, resultado->ai_addrlen))
        ERROR("bind()");

    return unsocket;
}
```

La primera línea define una macro llamada `ERROR` que nos ahorrará repetir un cierto código en caso de que funciones como `getaddrinfo()`, `bind()` o `recvfrom()` devuelvan un error. También hemos usado esta macro en la función `main()` con el mismo fin: generar una excepción en caso de que algo no vaya bien.

La función de configuración debe establecer en qué IP local y puerto quedará a la escucha el servidor. Con este fin preparamos la variable `conf` de tipo estructura `addrinfo`. Observa qué valores asignamos a los campos de la misma:

- **AI_PASSIVE**: pide a la función `getaddrinfo()` que rellene la dirección local por nosotros, en lugar de que la establezcamos de forma manual.
- **AF_UNSPEC**: en lugar de indicar que queremos trabajar con IPv4 o IPv6, con este valor se deja libertad de forma que podrían conectar clientes con una versión del protocolo u otra.
- **SOCK_DGRAM**: ya sabemos que esta constante designa el tipo de *socket* y, en este caso, determinaría que el protocolo a usar es UDP.

La función `getaddrinfo()` nos devolverá una lista de posibles direcciones IP locales en las que podríamos enlazar nuestro *socket* de servidor. Por esta razón se invoca a la función `socket()` dentro de un bucle, avanzando en la lista de direcciones enlazadas hasta que la creación del *socket* tenga éxito. En cada iteración se usan como parámetros los datos que hay en la variable `resultado`, devuelta antes por `getaddrinfo()`.

Nota

Revisa la práctica sobre programación con *sockets* relativa a direcciones para recordar cómo usar la función `getaddrinfo()`.

La función de configuración termina devolviendo el *socket*, que ya está preparado para comenzar a aceptar peticiones. Esa es la tarea que se completa en el bucle de la función `main()`.

Función de notificación de paquete recibido

Cada vez que la función `recvfrom()`, invocada desde `main()`, devuelve un segmento UDP recibido desde otro *host*, se ejecuta el código mostrado a continuación:

```
void informa_paquete(char paquete[], struct sockaddr_storage& dircliente) {
    char strdireccion[NI_MAXHOST + 1];

    inet_ntop(dircliente.ss_family,
              dircliente.ss_family == AF_INET ?
                (void *)&((struct sockaddr_in *) &dircliente)->sin_addr :
                (void *)&((struct sockaddr_in6 *) &dircliente)->sin6_addr,
              strdireccion, NI_MAXHOST);

    cout << "Recibidos " << strlen(paquete) << " bytes desde "
          << strdireccion << endl
          << "'" << paquete << "'" << endl;
}
```

Como parámetro se recibe la estructura `sockaddr_storage` devuelta por la función `recvfrom()`. Ahí encontramos, entre otros datos, la dirección IP desde la que conecta el cliente. Esta podría ser IPv4 o IPv6, razón por la que es preciso comprobar el campo `ss_family` antes de llevar a cabo la conversión. Además de la dirección, también se informa por la consola sobre el tamaño del paquete de datos recibido y se muestra su contenido.

Nota

La forma en que llamamos aquí a `inet_ntop()` es muy similar a la que aparece en uno de los ejercicios de la práctica previa de programación con *sockets*.

Archivos de cabecera

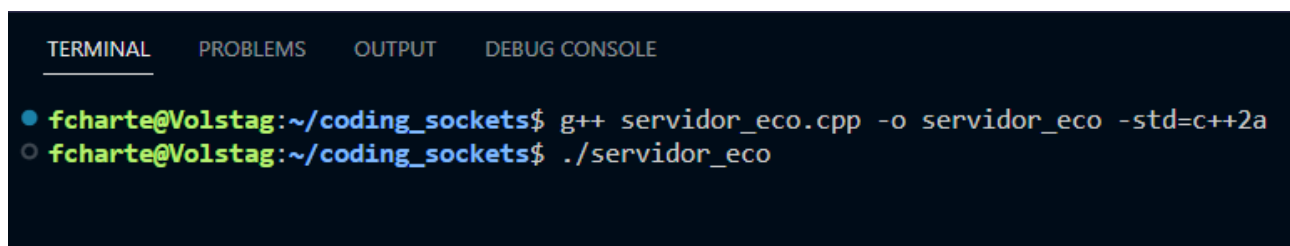
La compilación de este programa requiere que introduzcamos todo el código anterior en un archivo con extensión `.cpp`, cuyas primeras líneas se encargarían de incluir los archivos de cabecera apropiados:

```
#include <iostream>
#include <cstring>
#include <sys/socket.h>
#include <netdb.h>
#include <unistd.h>
#include <netinet/ip.h>
#include <arpa/inet.h>

using namespace std;
```

Compilación y ejecución del servidor

Ahora estamos en disposición de compilar y ejecutar el servidor, siguiendo los pasos que ya conocemos de las anteriores prácticas de programación con *sockets*.



```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

● fcharte@Volstag:~/coding_sockets$ g++ servidor_eco.cpp -o servidor_eco -std=c++2a
○ fcharte@Volstag:~/coding_sockets$ ./servidor_eco
```

Figura 1. Compilamos y ejecutamos el servidor de eco.

Tras lanzar la ejecución, el programa quedará a la espera de manera indefinida en la llamada a la función `recvfrom()`. Por tanto, necesitamos un cliente, algún programa, que envíe segmentos UDP a este servidor para poder verificar su funcionamiento.

EJERCICIO 1

Consulta la documentación del comando `nc` (*Netcat*) de Linux, en concreto la forma de actuar como cliente UDP. Usa dicho comando para enviar, desde una segunda ventana de terminal, algunos mensajes al servidor y observar su funcionamiento.

1. ¿Qué opciones has necesitado usar para enviar segmentos UDP con `nc`?
2. ¿Puedes usar `nc` para obtener una funcionalidad similar a la que ofrece este programa servidor?

Desarrollo de un cliente UDP

Ahora que tenemos el servidor en funcionamiento, el paso siguiente sería escribir un programa que actúe como cliente. Este se limitará a enviar el mensaje que se facilite por consola, mostrando a través de la misma la respuesta que se reciba. Como en casos previos, abrimos el módulo de código e introducimos los archivos de cabecera, macros y otras declaraciones:

```
#include <iostream>
#include <cstring>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <arpa/inet.h>

using namespace std;

#define ERROR(funcion) throw runtime_error(funcion + string(strerror(errno)))
```

Creación del *socket*

Todo el código del cliente estará alojado en la función `main()`. Las primeras sentencias de esta se encargarán de crear el *socket* que nos servirá para enviar un datagrama al servidor:

```
int main(int argc, char *argv[]) {
    const auto PUERTO_ECO = 32768;

    int socket_servidor = socket(AF_INET, SOCK_DGRAM, 0);
    if(socket_servidor == -1) ERROR("socket()");
```

Indicamos el tipo de direcciones que usaremos, con la constante `AF_INET`, así como el tipo de *socket*, que estará asociado al protocolo UDP que es el asociado a `SOCK_DGRAM`.

Envío de un segmento

Creado el *socket*, se prepara en una estructura `sockaddr_in` la dirección y puerto del *host* destinatario al que quiere enviarse el segmento UDP. La IP ha de ser facilitada como primer parámetro al invocar al programa (`argv[1]`). El contenido de ese segmento será un texto, a facilitar como segundo argumento desde la línea de comandos (`argv[2]`). Toda esta información se facilita a la función `sendto()`, tal y como se aprecia a continuación, y se comprueba que el valor devuelto no denote un error:

```
struct sockaddr_in destino {
    .sin_family = AF_INET,
    .sin_port = htons(PUERTO_ECO),
};
inet_aton(argv[1], &destino.sin_addr);
socklen_t longdestino = sizeof destino;
```



```
if(sendto(socket_servidor, argv[2], strlen(argv[2]), 0,
    (struct sockaddr *) &destino, sizeof destino) == -1)
    ERROR("sendto()");
```

Recepción de la respuesta

Tras enviar el segmento UDP, el cliente debe esperar la respuesta del servidor. Esta llegará desde la misma dirección y puerto al que se ha hecho el envío, por lo que serán esos datos los que se faciliten a la función `recvfrom()` tal y como se aprecia en el siguiente código:

```
char *respuesta = new char[IP_MAXPACKET];
auto bytes = recvfrom(socket_servidor, respuesta, IP_MAXPACKET, 0,
    (struct sockaddr *) &destino, &longdestino);
if(bytes == -1) ERROR("recvfrom()");

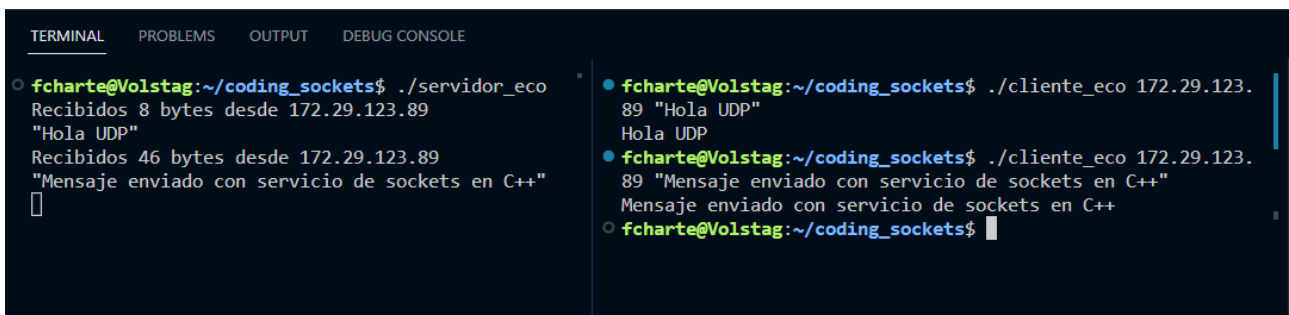
respuesta[bytes] = '\0';

cout << respuesta << endl;
}
```

La citada función nos devuelve como resultado el número de bytes efectivamente recibidos, que almacenamos en la variable `bytes`. Esos datos están alojados, en forma de caracteres, en el vector `respuesta`. Agregamos un byte `'\0'` final para asegurar que el envío de la cadena a la consola mostrará solo el contenido del paquete de respuesta, obviando cualquier contenido previo que dicho vector pudiese contener.

Prueba de comunicación entre cliente y servidor

Compilado el cliente, y asumiendo que tenemos el servidor en ejecución en otra terminal, no tenemos más que facilitarle como entrada la IP de destino y el mensaje. El resultado debería ser que el servidor notifica cada segmento recibido, mientras que el cliente confirma que ha recibido el eco de sus envíos, tal y como se aprecia en la figura 2.



```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

fcharte@Volstag:~/coding_sockets$ ./servidor_eco
Recibidos 8 bytes desde 172.29.123.89
"Hola UDP"
Recibidos 46 bytes desde 172.29.123.89
"Mensaje enviado con servicio de sockets en C++"
[]

fcharte@Volstag:~/coding_sockets$ ./cliente_eco 172.29.123.
89 "Hola UDP"
Hola UDP
fcharte@Volstag:~/coding_sockets$ ./cliente_eco 172.29.123.
89 "Mensaje enviado con servicio de sockets en C++"
Mensaje enviado con servicio de sockets en C++
fcharte@Volstag:~/coding_sockets$
```

Figura 2. Cliente y servidor comunicándose mediante UDP.

Desarrollo de un servidor y cliente de chat

Tomando como base el código de los dos anteriores programas, crearemos un **servicio básico de chat**. Para ello será necesario definir un protocolo simple, mediante el cual los clientes puedan comunicarse con el servidor. Todo el intercambio de información se realizará mediante segmentos UDP. Una vez definido el protocolo, llegará el momento de introducir en el código, tanto del servidor como del cliente, los cambios indicados en los respectivos ejercicios.

El protocolo de comunicación

Cualquier nuevo servicio puede definir un **protocolo de aplicación a su medida**. Es lo que haremos en este caso para nuestro servicio de *chat*: establecer qué mensajes existen, quién los envía y cuál es su finalidad. Este protocolo simple se compondrá de tres mensajes, siempre emitidos por los clientes y recibidos por el servidor, que serán los siguientes:

- **HELO usuario**: lo enviará un cliente que quiere conectarse al servicio. El servidor agregará al nuevo **usuario** a la lista de personas conectadas y notificará su llegada a todos ellos.
- **QUIT usuario**: enviado cuando el cliente se desconecta del servicio, a fin de que el servidor lo elimine de su lista de conectados y notifique la salida a todos los demás.
- **SEND mensaje**: permite a cualquier cliente conectado enviar un mensaje a todos los demás. El servidor se encargará de reenviar el mensaje a los usuarios que haya conectados en cada momento.

En cuanto a la recepción de datos por parte de los clientes, estos se limitarán a **mostrar por consola todo** aquello que reciban del servidor, sin necesidad de interpretar ningún comando. La Figura 3 es una representación del flujo de comunicación entre el servidor y dos clientes.

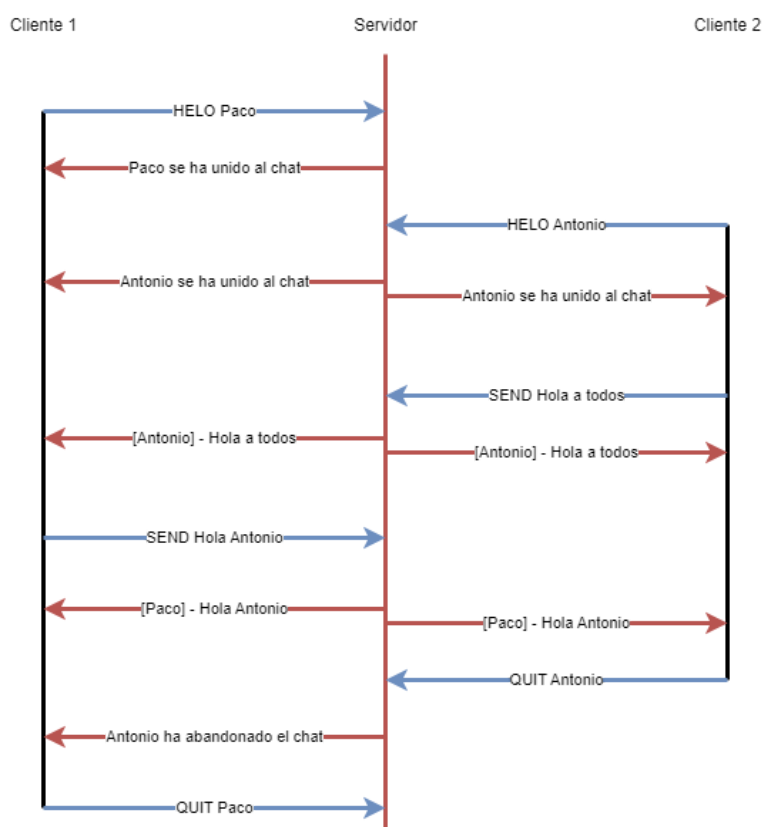


Figura 3. Cronograma con la comunicación entre el servidor y dos clientes.

Implementación del servidor

EJERCICIO 2

Tomando como punto de partida el código del servidor de eco introduce los cambios necesarios, atendiendo a las siguientes indicaciones, para implementar tu servidor de *chat*:

1. Elimina la función `informa_paquete()` e introduce en su lugar la nueva función `procesa_mensaje()`. Esta será invocada desde `main()` con cada segmento UDP recibido. Esta nueva función realizará las tareas descritas a continuación:
 - Contar con un vector o lista que permita almacenar tanto el identificador de cada usuario conectado como la información necesaria para enviarle segmentos UDP (su dirección IP y su puerto).
 - Separar el mensaje recibido en dos porciones: una cadena de cuatro caracteres que contendrá el **comando** y el resto de la cadena serán los **datos asociados** al mismo.
 - Según el comando recibido realizar el trabajo correspondiente, a saber:
 - a. **HELO**: **agregar al vector** el identificador del nuevo usuario y resto de datos. **Notificar a todos** los conectados quién se ha unido al *chat*. Para ello hay que recorrer todos los elementos del vector y enviar un segmento UDP a cada cliente.
 - b. **SEND**: obtener el identificador del usuario que envía el mensaje, **buscando en el vector** su IP/puerto, para componer un segmento que contenga `[usuario] -> mensaje`, siendo `mensaje` los datos recibidos con el comando **SEND**. Este segmento se enviará a todos los usuarios conectados.
 - c. **QUIT**: **eliminar del vector** el elemento que corresponde al identificador del usuario. **Notificar a todos** los conectados quién ha abandonado el *chat*.
 - Con fines de verificación, el servidor mostrará por consola una línea por cada mensaje procesado indicando, al menos, su origen y datos asociados.
2. **Compila y ejecuta** el nuevo servidor.
3. Usa la utilidad `nc` del Ejercicio 1 para comunicarte con él, enviando los distintos comandos para **comprobar su funcionamiento**. Puedes abrir varias terminales y usar `nc` desde cada una de ellas para simular conexiones de múltiples usuarios.

Implementación del cliente

EJERCICIO 3

Tomando como punto de partida el código del cliente de eco, desarrolla tu propio cliente de *chat* atendiendo a las siguientes indicaciones:

1. El programa recibirá **desde la línea de comandos** dos argumentos: la dirección IP del servidor de *chat* y el identificador de usuario con el que nos queremos unir al servicio.
2. Al iniciarse, el programa usará el comando **HELO** para unirse al servicio de *chat*. Esta será una **acción automática**, la llevará a cabo el cliente en cuanto inicie su ejecución, no una opción que deba efectuar el usuario.
3. A continuación el programa entrará en un **bucle infinito** con las siguientes operaciones en cada ciclo:
 - Recepción de un segmento desde el *socket* que se usó para enviar el comando **HELO** al servidor.
 - Impresión en la consola de la cadena recibida del servidor.
 - Obtención de una cadena de texto desde la entrada estándar (el teclado).
 - Preparación de un segmento que contenga el comando **SEND** seguido de dicha cadena y envío del mismo al servidor.
4. El programa deberá, al detectar la pulsación de la combinación ^C , enviar al servidor el comando **QUIT** y **finalizar sin esperar** respuesta. Para hacer esto necesitarás consultar la ayuda de la función `signal()` y la constante **SIGINT**.
5. Compila el programa y, asegurándote de que el servidor está en funcionamiento, ejecútalo desde varias ventanas de terminal, uniéndote al *chat* con **distintos nombres de usuario**. También puedes ejecutar el programa en varias máquinas distintas, ya sean físicas o virtuales.
6. ¿Qué problemas detectas en el funcionamiento del servicio? **Descríbelos** de manera concisa junto con **propuestas** que creas que podrían solucionarlos.

Mejora: en el archivo de cabecera `thread` está definida la clase `jthread`, parte de la biblioteca estándar de C++ y cuyo uso es muy sencillo. Basta con crear una variable de tipo `jthread`, facilitando como argumento el nombre de una función, para conseguir que una porción de código se ejecute en un hilo separado. Usando dicha clase, independizar la recepción de segmentos desde el servidor de la lectura desde el teclado para conseguir un funcionamiento más adecuado del servicio.

The image shows four terminal windows running on a Linux system. The top-left window is the chat server, and the other three are clients. The server logs the connections of Thor, Sif, and Loki, and relays their messages. The clients show the messages they receive from the server.

```

usuario@par: ~
Archivo Acciones Editar Vista Ayuda
usuario@par: ~ x
usuario@par:~$ ./servidor_chat
[Sistema] -> Thor se ha unido a la conversación
[Sistema] -> Sif se ha unido a la conversación
[Sif] -> Hola Thor
[Thor] -> Bienvenida Sif
[Sistema] -> Loki se ha unido a la conversación
[Thor] -> Hola hermano
[Loki] -> ¿Qué tal Thor?
[Sif] -> Hola Loki
[ ]

usuario@par:~$ ./cliente_chat 127.0.0.1 Thor
[Sistema] -> Thor se ha unido a la conversación
[Sistema] -> Sif se ha unido a la conversación
[Sif] -> Hola Thor
Bienvenida Sif
[Thor] -> Bienvenida Sif
[Sistema] -> Loki se ha unido a la conversación
Hola hermano
[Thor] -> Hola hermano
[Loki] -> ¿Qué tal Thor?
[Sif] -> Hola Loki
[ ]

usuario@par:~$ ./cliente_chat 10.0.2.15 Sif
[Sistema] -> Sif se ha unido a la conversación
Hola Thor
[Sif] -> Hola Thor
[Thor] -> Bienvenida Sif
[Sistema] -> Loki se ha unido a la conversación
[Thor] -> Hola hermano
[Loki] -> ¿Qué tal Thor?
Hola Loki
[Sif] -> Hola Loki
[ ]

usuario@par:~$ ./cliente_chat 10.0.2.15 Loki
[Sistema] -> Loki se ha unido a la conversación
[Thor] -> Hola hermano
¿Qué tal Thor?
[Loki] -> ¿Qué tal Thor?
[Sif] -> Hola Loki
[ ]

```

Figura 4. El servidor de chat en funcionamiento con tres clientes.

Material complementario

Al enviar y recibir datos mediante *sockets*, con funciones como `recvfrom()`, `sendto()`, `recv()` y `send()`, por regla general siempre será preciso realizar algún tipo de conversión. Hemos de tener presente que esas funciones deben permitir enviar cadenas de caracteres, números, ya sea enteros o en punto flotante, o cualquier otro tipo de información, como podría ser un archivo con una imagen. Esa es la razón de que se recurra al tipo `void *`, un puntero genérico adecuado para cualquier tipo de información.

En nuestros programas, sin embargo, usaremos vectores `char []` o el tipo `string` para trabajar con cadenas de caracteres, el tipo `int` para números enteros, el tipo `bool` para booleanos, etc. Al enviar este tipo de datos tendremos que convertir a `void *`. Análogamente, al recibir datos habrá que convertir de `void *` al tipo que nos interese. Esta es una tarea básica de trabajo con punteros, no específica de esta asignatura ni de la programación con *sockets*, por lo que si ya sabes cómo hacerlo no necesitarás estudiar este material complementario.

Conversión de vectores de caracteres

Para enviar una información que tenemos almacenada en un vector `char []`, o bien un puntero `char *`, no hay más que facilitar la variable a la función `sendto()/send()` indicando la conversión: `(void *)variable`. De hecho, la conversión implícita permite entregar a dichas funciones un parámetro de tipo `char []` o `char *` sin siquiera indicar explícitamente esa conversión, tal y como hemos hecho en algunos de los ejercicios de este guion.

Al indicar la longitud del dato a enviar, y asumiendo que se desee enviar la cadena completa, lo habitual es emplear la función `strlen(variable)`. De esta forma se entrega a la función toda la secuencia de caracteres, pero en ella no se incluye el byte `\0` que marca el final de la cadena. Si quisiéramos enviarlo también, como tamaño del dato a transferir habría que usar `strlen(variable) + 1`.

Cuando se recibe un dato, con `recvfrom()/recv()`, el parámetro a entregar debe contar con espacio suficiente para almacenar la información que nos llega. Por ello solemos hacer:

```
char *respuesta = new char[IP_MAXPACKET];
auto bytes = recvfrom(socket_servidor, respuesta, IP_MAXPACKET, ... );
```

Alternativamente podría declararse `respuesta` como `char respuesta[IP_MAXPACKET]` y usar asignación de memoria estática, en lugar de dinámica. El resultado sería el mismo y, además, no tendríamos que liberar la memoria reservada con `new`.

La función de recepción devuelve como valor un entero que indica la cantidad de bytes recibidos. Según como se haya hecho el envío, en ese bloque de bytes estará incluido o no el byte `\0` de finalización de la cadena, según lo explicado antes. Siempre es posible añadirlo en este momento, tras la recepción, como también se ha mostrado en los ejercicios de este guion.

Conversion de cadenas tipo `string`

Es probable que estemos más habituados a trabajar con el tipo `string`, a la hora de tratar cadenas de caracteres, que con vectores o punteros `char`. El tipo `string`, no obstante, puede convertirse fácilmente a dichos tipos, lo cual facilita su transmisión con las funciones mencionadas.

Si tenemos una cadena en una variable `string`, para obtener el correspondiente puntero de tipo `char` recurriremos al método `c_str()`. La longitud de la cadena se obtendría con el método `length()`. De esta forma, para enviar una variable `string` haríamos:

```
string cadena;

sendto(socket, cadena.c_str(), cadena.length(), ...)
```

También podríamos convertir la variable al tipo `char *`, con la expresión `char * cadena2 = cadena.c_str()`, y a partir de ahí trabajar como se ha explicado en el anterior apartado.

Para recibir una cadena de caracteres y trabajar con ella como tipo *string*, lo más fácil es usar un vector o puntero *char*, como se ha explicado antes, ya de que esta forma se reserva el espacio necesario para guardar la información recibida. A continuación, no hay más que convertir esa variable a *string*:

```
string mirespuesta;  
char *respuesta = new char[IP_MAXPACKET];  
auto bytes = recvfrom(socket_servidor, respuesta, IP_MAXPACKET, ... );  
  
mirespuesta = respuesta;
```

Una vez se tiene la información en la variable *string*, usaríamos los métodos de esta para operar sobre la cadena y, por ejemplo, quedarnos con los primeros N caracteres, buscar una subcadena, etc., según la funcionalidad que vayamos a implementar. A continuación se enumeran algunos de los métodos de la clase *string* para manipular subcadenas y realizar búsquedas:

- *substr(pos, len)*: Devuelve una nueva cadena que es una subcadena de la cadena original.
 - *pos*: índice del primer carácter de la subcadena.
 - *len*: longitud de la subcadena.
- *substr(pos)*: devuelve una subcadena desde la posición *pos* hasta el final de la cadena.
- *find(s)*: busca la primera aparición de la subcadena *s* dentro de la cadena y devuelve el índice de su posición inicial. Si no se encuentra, devuelve *std::string::npos*.
- *rfind(s)*: busca la última aparición de la subcadena *s* dentro de la cadena y devuelve el índice de su posición inicial. Si no se encuentra, devuelve *std::string::npos*.
- *find_first_of(s)*: busca el primer carácter de la cadena que está presente en la cadena *s* y devuelve su posición.
- *find_last_of(s)*: busca el último carácter de la cadena que está presente en la cadena *s* y devuelve su posición.

Ejemplo de uso

```
#include <iostream>  
#include <string>  
  
int main() {  
    std::string str = "Hola, mundo!";  
  
    // Obtener subcadena  
    std::string subStr = str.substr(3, 5); // "a, mu"  
    std::cout << subStr << std::endl;  
  
    // Buscar subcadena
```

```

    auto pos = str.find("mundo");
    if (pos != std::string::npos) {
        std::cout << "La palabra 'mundo' se encuentra en la posición: " <<
pos << std::endl;
    }

    // Buscar el primer carácter que es una vocal
    pos = str.find_first_of("aeiou");
    std::cout << "El primer carácter vocal es: " << str[pos] << std::endl;

    return 0;
}

```

Conversión de tipos de datos numéricos

Las variables de tipo numérico, ya sea entero o flotante, así como el tipo `bool`, tienen un tamaño fijo indistintamente del valor que contengan. Dicho tamaño se obtiene con el operador `sizeof`. Si lo que nos interesa es enviar un valor de tipo `int`, por poner un ejemplo, podemos declarar una variable `char *` y, a continuación, hay dos alternativas:

1. Hacer que el puntero `char *` apunte a la zona de memoria donde está el entero, de forma que ambas variables, la de tipo `int` y la de tipo `char *`, facilitan dos vistas diferentes de la misma información, sin duplicarla.
2. Reservar el espacio que se precisa para guardar un entero y copiar el contenido de la variable `int` a la variable `char *`

En general, preferiremos el primer método de forma que siempre exista un único valor y no haya necesidad de copiar entre la variable numérica y la cadena de caracteres:

```

int edad = 67;
char * cedad;

cedad = (char *)&edad; // Ahora cedad se puede enviar con sendto() como
cualquier cadena

```

La conversión en sentido inverso, suponiendo que hemos recibido algo con `recvfrom()` y lo tenemos guardado, no precisa más que una conversión de tipo como la siguiente:

```

int valor;

valor = *(int *) buffer; // buffer es una variable void* o char* con la
información recibida

std::cout << valor << std::endl;

```


Nota: otra vía para enviar y recibir datos numéricos consiste en convertirlos a una cadena de caracteres u obtenerlos desde una cadena de caracteres. El primer paso se efectúa mediante el método `to_string()`. Este acepta como único argumento un número, de tipo entero o punto flotante, y devuelve un `string` sobre el que ya sabemos cómo trabajar. El paso inverso, obtener un número desde un `string`, se consigue mediante funciones como `stoi()`, `stol()`, `stof()`, etc., según que el número sea `int`, `long`, `float` y así sucesivamente.